KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

Git 기본 흐름, 눈으로 보기

add, commit, push, pull 할 때
내 파일은 어디로 갈까요?



오늘의 순서

1	개념 잡기	4개의 공간, 파일 상태와 용어	3–8쪽
2	혼자 기록하기	init, add, commit, 커밋 메시지 규칙	9–18쪽
3	GitHub과 연결	remote add, push	19–23쪽
4	함께 작업하기	clone, fetch, pull	24–28쪽
5	되돌리기	reset, revert, reflog	29–35쪽
6	막힐 때	status 읽기, 자주 나는 에러	36–43쪽

44쪽은 한 장 요약이에요. 2부부터 4부까지는 hello-git 저장소 하나로 이어서 실습해요.

왜 Git을 쓸까요?

파일은 하나로 두고, 기록을 쌓아요

Git 없이

보고서_최종.docx

보고서_최종_수정.docx

보고서_진짜최종.docx

보고서_진짜최종_v3.docx

어느 게 최신이지?

Git과 함께

보고서.docx

- **e73617d** 팀장님 피드백 반영
- **a7c21f4** 오타 수정
- **538dd06** 첫 초안

원하는 시점으로 돌아갈 수 있어요

Git에는 4개의 공간이 있어요

내 컴퓨터

① 작업 폴더

내가 고치는 파일

② 대기실

커밋할 것만 골라 둠

③ 내 저장소

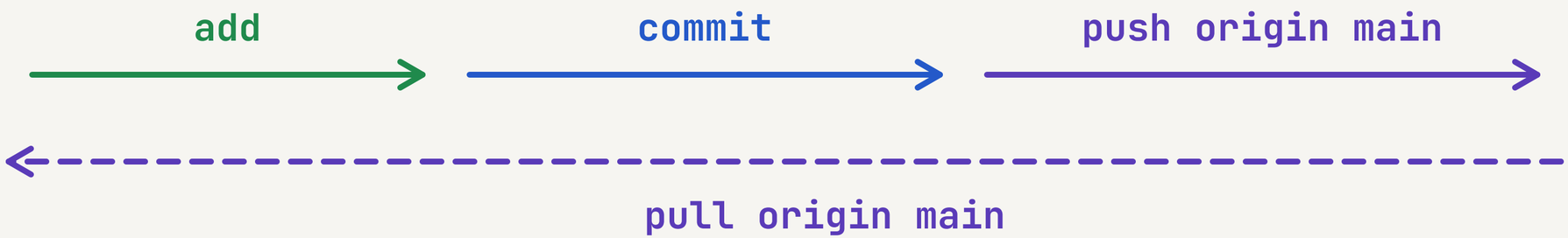
커밋이 쌓이는 곳

인터넷

④ 원격 저장소

GitHub, 팀과 공유

②와 ③은 숨김 폴더 .git 안에 있어요



택배로 비유하면

택배를 보내는 순서와 같아요



`add`

상자에 담기

`commit`

봉하고
송장 붙이기

`remote add`

받는 주소
등록

`push`

발송

`pull`

도착한 택배
받아서 풀기

커밋 메시지 = 송장　　**커밋 해시** = 운송장 번호

용어 ① 파일의 상태

git status 글씨 색으로 상태를 알아요

상태	뜻	git status에 보이는 말
Untracked	처음 보는 새 파일	`Untracked files:`
Modified	고쳤지만 add 전	`Changes not staged for commit:`
Staged	대기실에 올라감	`Changes to be committed:`
Committed	마지막 커밋과 같음	`working tree clean`

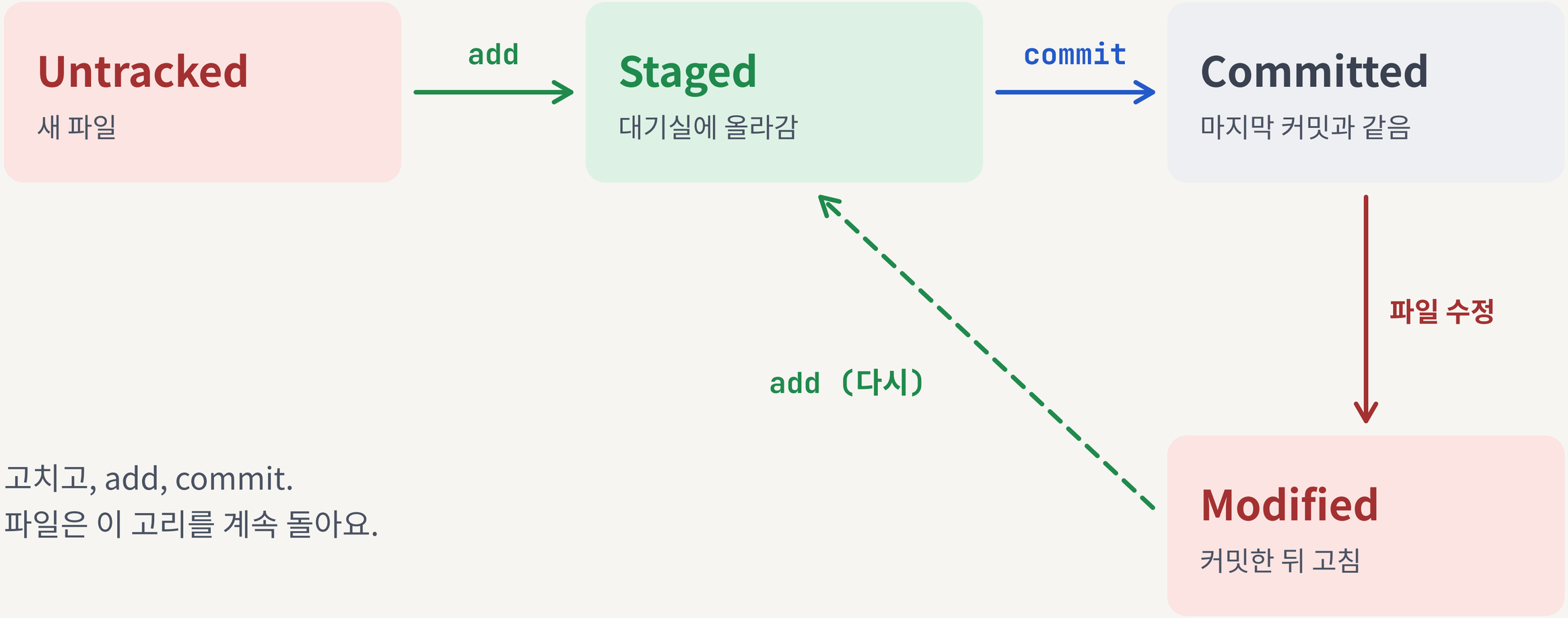
빨간 글씨는 대기실 밖, 초록 글씨는 대기실 안이에요.

용어 ② 저장소와 이름표

origin은 명령어가 아니라 주소의 별명이에요

용어	한 줄 뜻
Repository	커밋 기록 보관함 (.git, GitHub)
Index	대기실의 정식 이름
Commit, 해시	저장된 한 시점과 그 고유 번호
branch (브랜치)	커밋에 붙이는 이름표. 03에서 자세히 봐요
main	기본 브랜치. 최신 커밋에 붙은 이름표
HEAD	내가 지금 서 있는 곳
origin	GitHub 주소의 별명
origin/main	내 컴퓨터가 기억하는 GitHub main 위치

파일 하나의 한살이



고치고, add, commit.
파일은 이 고리를 계속 돌아요.

실습 0 · 컴퓨터마다 처음 한 번만

Git에게 내 이름을 알려 줘요

```
$ git config --global user.name "홍길동"                    ← 커밋마다 붙는 작성자 이름
$ git config --global user.email "gildong@example.com"    ← GitHub 가입 메일과 같게
$ git config --global init.defaultBranch main             ← 기본 브랜치 이름을 main으로

$ git config --global --list
user.name=홍길동
user.email=gildong@example.com
init.defaultbranch=main
```

--global은 이 컴퓨터 전체에 적용돼요. 그래서 한 번만 하면 돼요.

실습 1 · 혼자 기록하기

폴더를 Git 저장소로 만들어요

내 컴퓨터

① 작업 폴더

빈 폴더

② 대기실

비어 있음

숨김 폴더 .git이 생겼어요.
②와 ③은 이 폴더 안에 있어요.

③ 내 저장소

커밋 없음

인터넷

④ GitHub

```
$ mkdir hello-git && cd hello-git
$ git init
Initialized empty Git repository in /Users/kim/hello-git/.git/
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

실습 2 · 혼자 기록하기

새 파일을 하나 만들어요

내 컴퓨터

인터넷

① 작업 폴더

hello.md

Untracked · 새 파일

Git이 처음 보는 파일

② 대기실

③ 내 저장소

④ GitHub

```
$ echo "hello" > hello.md
$ git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.md
nothing added to commit but untracked files present
```

실습 3 · 혼자 기록하기

파일을 대기실에 올려요

내 컴퓨터

① 작업 폴더

hello.md
원본은 그대로

그 순간의 사진이 올라가요

② 대기실

hello.md
Staged · 새 파일

③ 내 저장소

인터넷

④ GitHub

```
$ git add hello.md
# 아무 출력이 없으면 성공
$ git status
On branch main
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.md
```

실습 4 · 혼자 기록하기

대기실에 올린 모습을 커밋해요

내 컴퓨터

인터넷

① 작업 폴더

hello.md

Committed · 변경 없음

② 대기실

비어 있음

③ 내 저장소

538dd06 main

538dd06 번으로 기록됐어요

④ GitHub

```
$ git commit -m "feat: 인사 파일 추가"
[main (root-commit) 538dd06] feat: 인사 파일 추가
 1 file changed, 1 insertion(+)
 create mode 100644 hello.md
$ git status
On branch main
nothing to commit, working tree clean
```

실습 5 · 혼자 기록하기

커밋한 파일을 다시 고쳐요

내 컴퓨터

① 작업 폴더

hello.md

Modified · 수정됨

마지막 커밋과 달라졌어요

② 대기실

③ 내 저장소

538dd06 main

인터넷

④ GitHub

```
$ echo "world" >> hello.md
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
        modified:   hello.md
no changes added to commit (use "git add" and/or "git commit -a")
```

실습 6 · 헷갈림 주의

add 한 뒤에 또 고치면 어떻게 될까요?

내 컴퓨터

① 작업 폴더

hello.md
Modified · more 추가

② 대기실

hello.md
Staged · world까지

같은 파일이 두 칸에 있어요.
지금 commit 하면 more는 빠져요.

③ 내 저장소

538dd06 main

인터넷

④ GitHub

```
$ git add hello.md
$ echo "more" >> hello.md
$ git status
Changes to be committed:
        modified:   hello.md
Changes not staged for commit:
        modified:   hello.md
# 안내 문구(use ...)는 생략
```

실습 7 · 혼자 기록하기

add, commit을 한 번 더 해요

내 컴퓨터

① 작업 폴더

hello.md

Committed · 변경 없음

② 대기실

비어 있음

③ 내 저장소

a7c21f4 **main**

538dd06

main은 늘 최신 커밋에 붙어요

인터넷

④ GitHub

```
$ git add hello.md
$ git commit -m "feat: 인사말 추가"
[main a7c21f4] feat: 인사말 추가
 1 file changed, 2 insertions(+)
$ git log --oneline
a7c21f4 (HEAD -> main) feat: 인사말 추가
538dd06 feat: 인사 파일 추가
```

커밋 메시지 ① · 방금 쓴 feat: 는 뭘까?

한 달 뒤, 어느 쪽 기록을 읽고 싶나요?

아무렇게나

```
$ git log --oneline
9f1c2aa 수정
4b7e0d1 ㅇㅇ
c21a9f3 asdf
7d0e5b2 최종
1a2b3c4 진짜최종
```

규칙대로

```
$ git log --oneline
9f1c2aa fix: 로그인 버튼이 안 눌리는 문제 수정
4b7e0d1 feat: 회원가입 이메일 인증 추가
c21a9f3 docs: README에 설치 방법 추가
7d0e5b2 style: 들여쓰기 정리
1a2b3c4 refactor: 로그인 함수 분리
```

- 무엇이 바뀌었는지 바로 보여요
- 팀원이 리뷰하기 쉬워요
- 문제 생긴 시점을 찾기 쉬워요

커밋 메시지 ②

타입, 콜론과 공백, 요약 순서로 써요

```
feat: 회원가입 이메일 인증 추가
```

타입

콜론과 공백

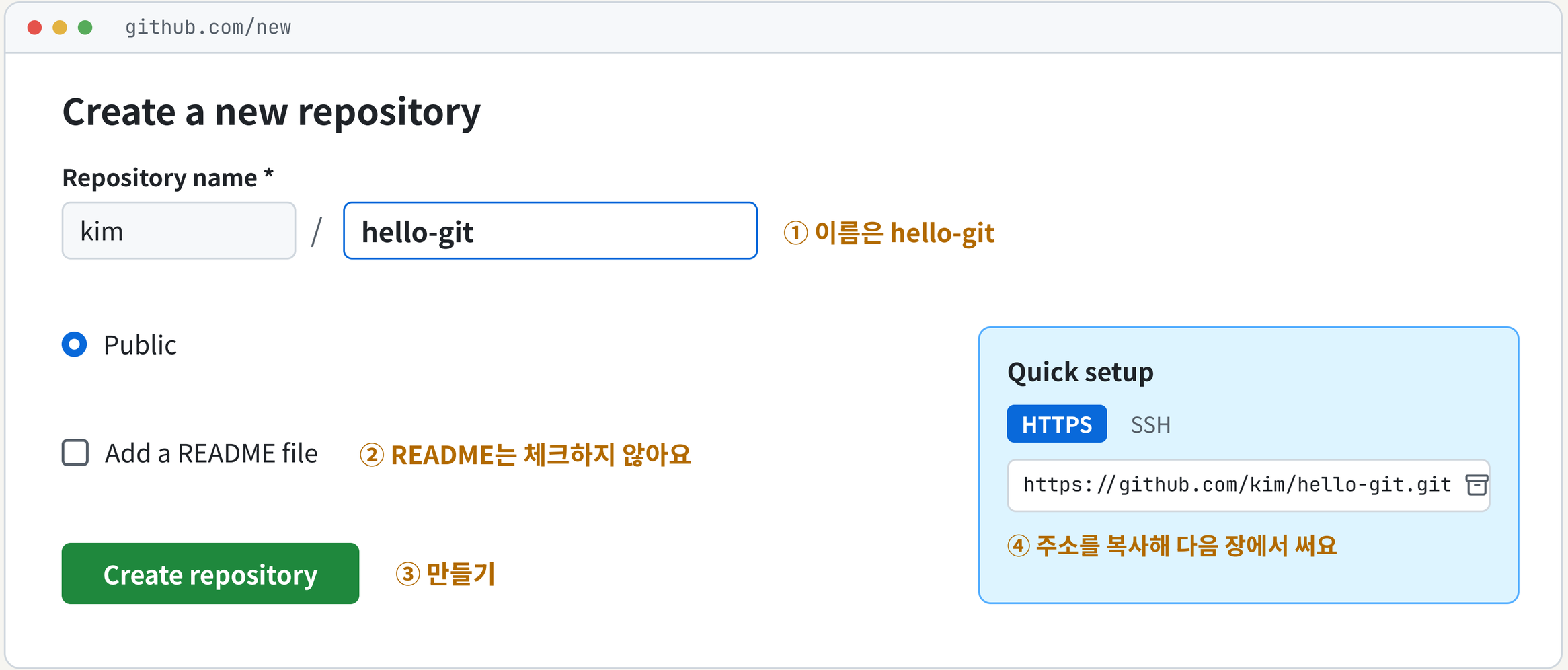
무엇을 했는지

feat	새 기능
fix	버그 수정
docs	문서 (README 등)
style	모양만 정리
refactor	동작은 같게, 구조 개선
test	테스트 코드
chore	설정, 패키지

한 커밋에 한 가지 일, 요약은 짧게

실습 8 · GitHub과 연결하기

GitHub에 빈 저장소를 만들어요



실습 9 · GitHub과 연결하기

GitHub 주소를 origin이라는 이름으로 등록해요

내 컴퓨터

① 작업 폴더

hello.md
Committed · 변경 없음

② 대기실

③ 내 저장소

a7c21f4 main
538dd06

인터넷

④ GitHub

빈 저장소

주소에 별명 origin을 붙였어요.
아직 보낸 건 없어요.

```
$ git remote add origin https://github.com/kim/hello-git.git
# 출력 없음 = 등록 성공
$ git remote -v
origin  https://github.com/kim/hello-git.git (fetch)
origin  https://github.com/kim/hello-git.git (push)
```

실습 10 · GitHub과 연결하기

내 커밋을 GitHub으로 보내요

내 컴퓨터

① 작업 폴더

hello.md

Committed · 변경 없음

② 대기실

③ 내 저장소

a7c21f4 main origin/main

538dd06

인터넷

④ GitHub

a7c21f4 main

538dd06

해시가 같으면 같은 커밋이에요

```
$ git push origin main
Enumerating objects: 6, done.
Writing objects: 100% (6/6), 466 bytes | 466.00 KiB/s, done.
To https://github.com/kim/hello-git.git
 * [new branch]      main -> main
$ git log --oneline
a7c21f4 (HEAD -> main, origin/main) feat: 인사말 추가
538dd06 feat: 인사 파일 추가
```

실습 11 · 커밋만 하고 push 안 하면?

status는 깨끗한데, GitHub에는 없어요

내 컴퓨터

① 작업 폴더

hello.md
Committed · 변경 없음

② 대기실

③ 내 저장소

3f9a2c1 **main**
a7c21f4 **origin/main**
538dd06

main이 origin/main보다 한 칸 위,
아직 안 보낸 커밋이에요

인터넷

④ GitHub

a7c21f4 **main**
538dd06

```
$ git add hello.md
$ git commit -m "feat: 끝인사 추가"
[main 3f9a2c1] feat: 끝인사 추가
$ git status
nothing to commit, working tree clean
$ git log --oneline
3f9a2c1 (HEAD -> main) feat: 끝인사 추가
a7c21f4 (origin/main) feat: 인사말 추가
538dd06 feat: 인사 파일 추가
```

실습 12 · GitHub과 연결하기

한 번 더 push 해서 맞춰요

내 컴퓨터

① 작업 폴더

hello.md
Committed · 변경 없음

② 대기실

③ 내 저장소

3f9a2c1 main origin/main
a7c21f4
538dd06

main과 origin/main이 같은 칸이에요

인터넷

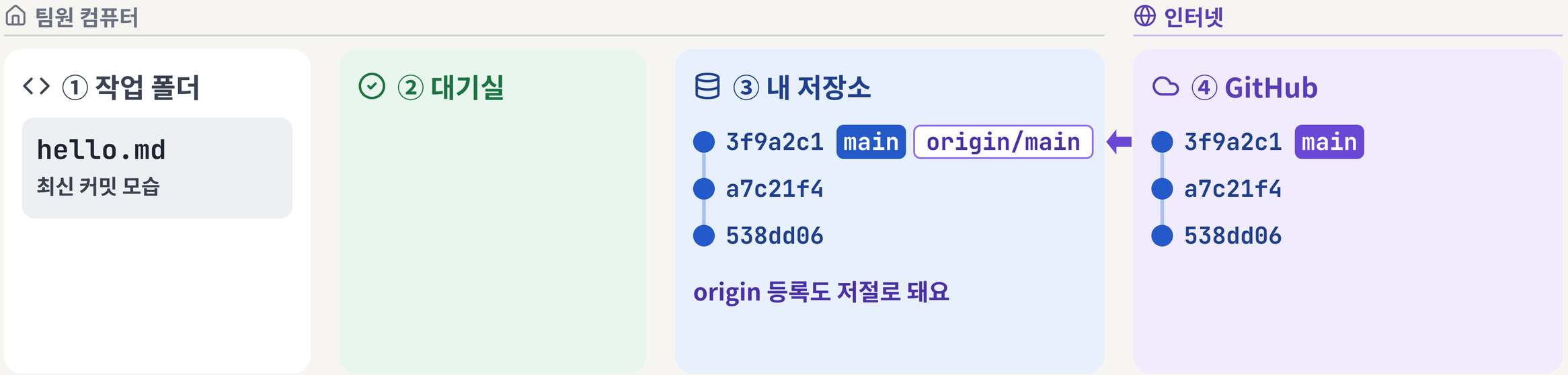
④ GitHub

3f9a2c1 main
a7c21f4
538dd06

```
$ git push origin main
To https://github.com/kim/hello-git.git
   a7c21f4..3f9a2c1  main -> main
$ git log --oneline
3f9a2c1 (HEAD -> main, origin/main) feat: 끝인사 추가
a7c21f4 feat: 인사말 추가
538dd06 feat: 인사 파일 추가
```

실습 13 · 팀원 컴퓨터에서

팀원이 저장소를 통째로 받아요




```
$ cd ~/projects
$ git clone https://github.com/kim/hello-git.git
Cloning into 'hello-git'...
Receiving objects: 100% (9/9), done.
$ cd hello-git
# 새로 생긴 폴더로 꼭 들어가기
$ ls
hello.md
```

실습 14 · 다시 내 컴퓨터 (팀원이 README를 push했어요)

GitHub 소식을 받아만 와요

내 컴퓨터

인터넷

① 작업 폴더

hello.md

Committed · 변경 없음

작업 폴더는 그대로예요

② 대기실

③ 내 저장소

e73617d origin/main

3f9a2c1 main

a7c21f4

538dd06

origin/main만 올라갔어요

④ GitHub

e73617d main

3f9a2c1

a7c21f4

538dd06

```
$ git fetch origin
From https://github.com/kim/hello-git
   3f9a2c1..e73617d  main       -> origin/main
$ git log --oneline origin/main
e73617d (origin/main) docs: README 추가
3f9a2c1 (HEAD -> main) feat: 끝인사 추가
a7c21f4 feat: 인사말 추가
538dd06 feat: 인사 파일 추가
```

실습 15 · 받아서 합치기

받아서 내 것에 합쳐요

내 컴퓨터

인터넷

① 작업 폴더

hello.md
Committed · 변경 없음

README.md
pull로 생김

② 대기실

③ 내 저장소

e73617d main origin/main
3f9a2c1
a7c21f4
538dd06

main이 origin/main까지 따라왔어요

④ GitHub

e73617d main
3f9a2c1
a7c21f4
538dd06

```
$ git pull origin main
From https://github.com/kim/hello-git
 * branch            main       -> FETCH_HEAD
Updating 3f9a2c1..e73617d
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

개념 정리

pull은 fetch와 merge를 한 번에 해요



① `git fetch origin`

받아만 와요

② `git merge origin/main`

내 것과 합쳐요

`git pull origin main` = ① + ② 한 번에

init과 clone

GitHub에 저장소가 있으면 clone, 없으면 init

GitHub에 아직 없으면

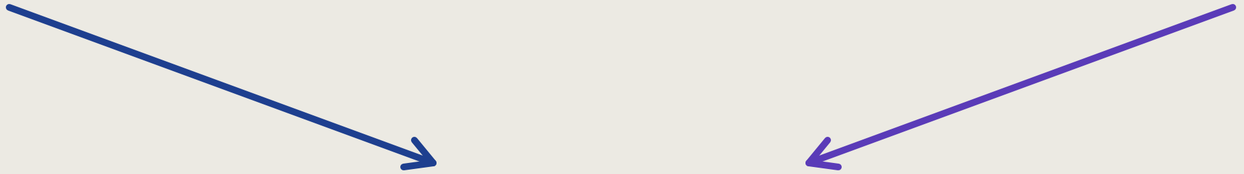
```
git init
```

```
git remote add origin 주소
```

GitHub에 이미 있으면

```
git clone 주소
```

```
cd 폴더이름
```

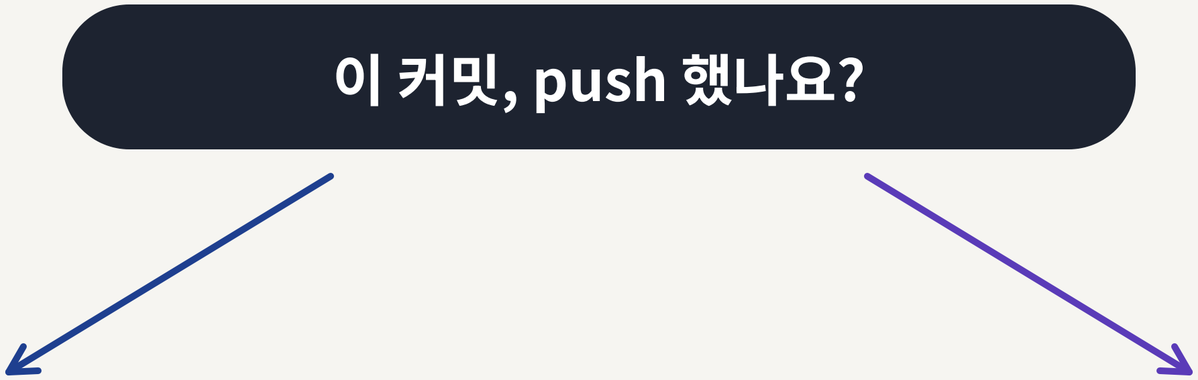


시작한 뒤에는 똑같아요

add → commit → git push origin main

되돌리기

커밋을 되돌릴 땐 push 했는지부터 봐요



아니요 · 내 저장소에만 있어요

```
git reset HEAD~1
```

커밋을 없던 일로 해요

잘못 지웠다면 git reflog로 다시 찾아요

네 · GitHub에도 올라갔어요

```
git revert 커밋
```

반대로 고친 커밋을 더해요

팀원 기록은 그대로 둬요

실습 16 · 되돌리기

앗, 잘못 커밋했어요

내 컴퓨터

① 작업 폴더

hello.md
Committed · 변경 없음

README.md
Committed · 변경 없음

② 대기실

③ 내 저장소

- b81d4e2 **main**
- e73617d **origin/main**
- 3f9a2c1
- a7c21f4
- 538dd06

인터넷

④ GitHub

- e73617d **main**
- 3f9a2c1
- a7c21f4
- 538dd06

GitHub에는 아직 없어요 (push 전)

```
$ echo "oops" >> hello.md
$ git add hello.md
$ git commit -m "feat: 실수한 커밋"
[main b81d4e2] feat: 실수한 커밋
 1 file changed, 1 insertion(+)
$ git log --oneline
b81d4e2 (HEAD -> main) feat: 실수한 커밋
e73617d (origin/main) docs: README 추가
```

실습 17 · 되돌리기 (push 전)

push 전이면 reset으로 커밋을 취소해요

내 컴퓨터

① 작업 폴더

hello.md
Modified · oops 남음

README.md
Committed · 변경 없음

② 대기실

③ 내 저장소

b81d4e2
e73617d main origin/main
3f9a2c1
a7c21f4
538dd06

main이 한 칸 내려왔어요

인터넷

④ GitHub

e73617d main
3f9a2c1
a7c21f4
538dd06

```
$ git reset HEAD~1
Unstaged changes after reset:
M       hello.md
$ git log --oneline
e73617d (HEAD -> main, origin/main) docs: README 추가
3f9a2c1 feat: 끝인사 추가
# HEAD~1 = 지금보다 한 칸 아래 커밋
```

되돌리기 · reset 옵션 3가지

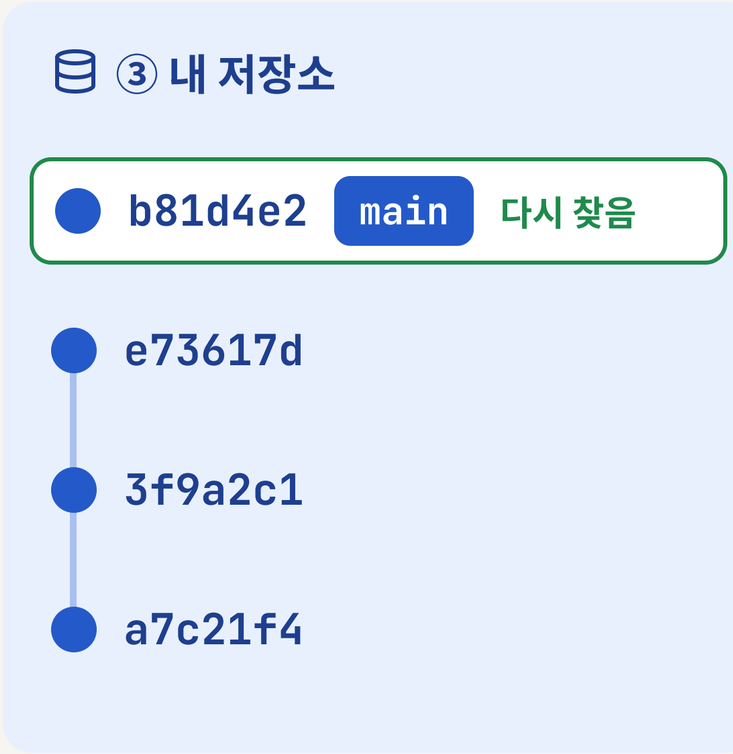
커밋한 변경은 어디로 갈까요?

옵션	③ 내 저장소	② 대기실	① 작업 폴더
`--soft`	커밋 취소	**변경이 여기 남음**	그대로
`--mixed` 옵션 없이 쓰면 이것	커밋 취소	비움	**변경이 여기 남음**
`--hard`	커밋 취소	비움	**변경이 사라짐**

처음엔 옵션 없이 git reset HEAD~1만 써요.

되돌리기 · 지운 커밋 다시 찾기

reset으로 지운 커밋, git reflog로 찾아요




```
$ git reflog
e73617d HEAD@{0}: reset: moving to HEAD~1
b81d4e2 HEAD@{1}: commit: feat: 실수한 커밋
e73617d HEAD@{2}: pull origin main: Fast-forward
3f9a2c1 HEAD@{3}: commit: feat: 끝인사 추가
```

```
$ git reset --hard b81d4e2
HEAD is now at b81d4e2 feat: 실수한 커밋
```

HEAD가 움직인 기록이 최신순으로 남아요.
reflog는 내 컴퓨터에만 있고 GitHub에는 없어요.

실습 18 · 만약 push까지 했다면?

push 한 커밋은 revert로 되돌려요

내 컴퓨터

① 작업 폴더

hello.md
Committed · 변경 없음

README.md
Committed · 변경 없음

② 대기실

③ 내 저장소

- c4e9a07 main origin/main
- b81d4e2
- e73617d
- 3f9a2c1
- a7c21f4

반대로 고친 c4e9a07을 더했어요

인터넷

④ GitHub

- c4e9a07 main
- b81d4e2
- e73617d
- 3f9a2c1
- a7c21f4

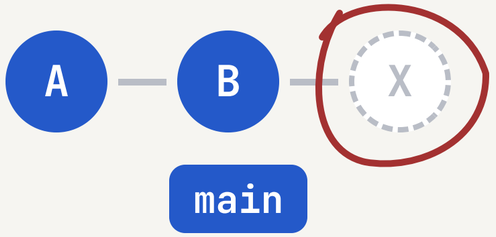
```
# b81d4e2(실수한 커밋)을 이미 push한 상태
$ git revert HEAD
[main c4e9a07] Revert "feat: 실수한 커밋"
 1 file changed, 1 deletion(-)
$ git push origin main
To https://github.com/kim/hello-git.git
   b81d4e2..c4e9a07  main -> main
```

되돌리기 · 정리

reset은 지우고, revert는 덧붙여요

reset

push 전, 나만 가진 커밋



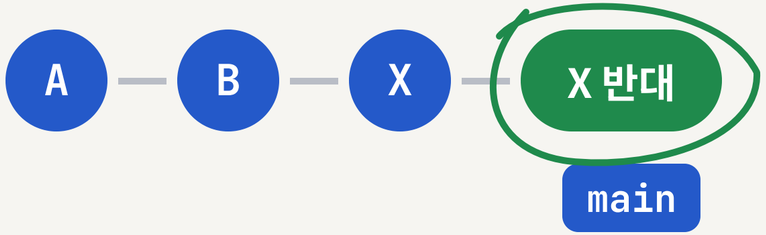
main을 B로 되감아요

X는 기록에서 빠져요

---

revert

push 후, 공유된 커밋



X를 지우지 않아요

반대로 고친 커밋을 더해요

막힐 때

막히면 git status부터 봐요

이 문장이 보이면	지금 상태	다음 할 일
`No commits yet`	커밋 0개	**add → commit**
`Untracked files:`	① 새 파일	**git add**
`Changes not staged for commit:`	① 고쳤지만 add 전	**git add**
`Changes to be committed:`	② 커밋 대기	**git commit**
`nothing to commit, working tree clean`	모두 커밋됨	**push 했는지 확인**

GitHub과 비교는 log로 해요

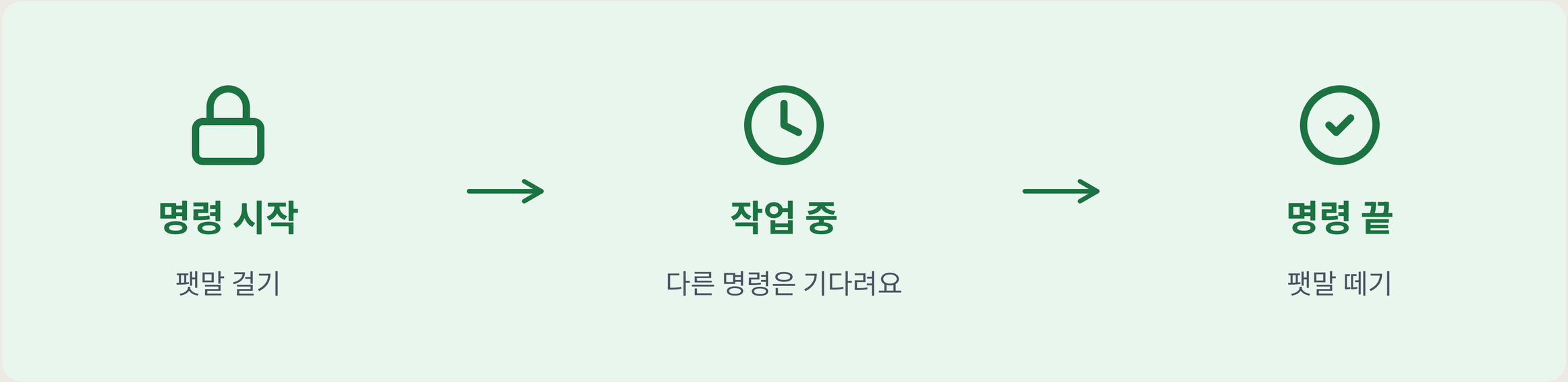
`git fetch origin → git log --oneline origin/main`

main이 위 → git push origin main

origin/main이 위 → git pull origin main

에러 1 · index.lock

index.lock은 Git의 '사용 중' 팻말이에요



끝나기 전에 멈추면 팻말이 남아요

터미널 창을 강제로 닫았을 때

커밋 메시지 에디터를 닫지 않았을 때

VS Code가 같은 저장소에서 git을 실행 중일 때

에러 1 · index.lock

이 메시지가 보이면 팻말만 떼요

```
$ git add hello.md
fatal: Unable to create '/Users/kim/hello-git/.git/index.lock': File exists.
Another git process seems to be running in this repository, ...
```

1

다른 git 작업이 끝났는지 봐요

커밋 에디터, 다른 터미널 창, VS Code

2

팻말을 떼요

```
rm .git/index.lock
```

Mac 터미널, Windows Git Bash 똑같아요

3

아까 명령을 다시 쳐요

```
git add hello.md
```

지우는 건 index.lock 하나뿐이에요. index나 .git 폴더는 지우지 않아요.

에러 2 · add 없이 commit

commit 했는데 아무 일도 없어요

```
$ git commit -m "feat: 새 파일"
Untracked files:
        new.md
nothing added to commit but untracked files present

$ git commit -m "fix: 오타 수정"
Changes not staged for commit:
        modified:   hello.md
no changes added to commit
```

② 대기실이 비어 있어요

commit은 대기실에 있는 것만 기록해요.

이렇게 고쳐요

```
git add 파일
```

```
git commit -m "..."
```

에러 3 · origin 설정

origin이 이미 있대요, 혹은 없대요

두 번 등록했어요

```
error: remote origin already exists.
```

remote add를 두 번 쳤거나
clone한 폴더에서 또 등록했어요

이렇게 고쳐요

```
git remote -v
```

```
git remote set-url origin 새주소
```

등록하지 않았어요

```
fatal: 'origin' does not appear
to be a git repository
```

init으로 시작하고
remote add를 빠뜨렸어요

이렇게 고쳐요

```
git remote add origin 주소
```

```
git push origin main
```

에러 4 · 어디로 보낼지 안 적음

git push만 쳤더니 멈춰요

```
$ git push
fatal: The current branch main has no upstream branch.
$ git pull
There is no tracking information for the current branch.
```

어디로, 무엇을 항상 적어요

```
git push origin main
git pull origin main
```

어디로 — 무엇을

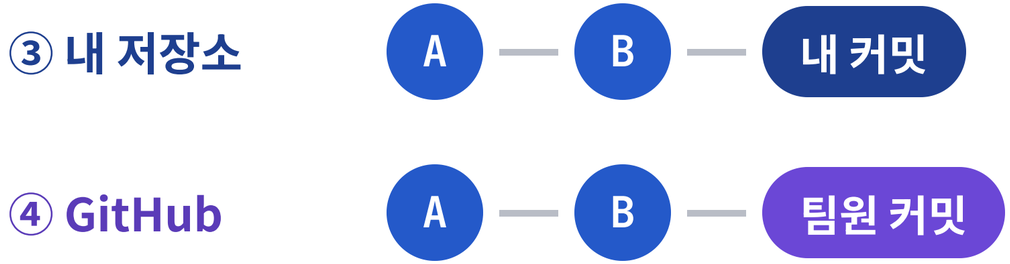
⚠

에러가 --set-upstream(-u)을 권해도 쓰지 않아요

에러 5 · push 거절

push 했더니 rejected가 나와요

```
$ git push origin main
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/kim/hello-git.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. ...
```



서로 모르는 커밋이 하나씩 생겼어요

받아서 합친 뒤 다시 push 해요

다음 장에서 순서대로 봐요

--force는 쓰지 않아요

에러 5 · push 거절

pull 설정을 한 번 해 두고 받아서 합쳐요

```
$ git pull origin main
From https://github.com/kim/hello-git
 * branch            main       -> FETCH_HEAD
hint: You have divergent branches and need to specify how to reconcile them.
fatal: Need to specify how to reconcile divergent branches.
$ git config --global pull.rebase false                    ← 처음 한 번만 쳐요 (03에서 다시 봐요)
$ git pull origin main
Merge made by the 'ort' strategy.                          ← vim이 열리면 :wq 치고 Enter
 README.md | 1 +
$ git push origin main
   dac2ff4..71f86b2  main -> main
```

한 장 요약

① 작업 폴더 | ② 대기실 | ③ 내 저장소 | ④ GitHub



새로 시작

```
git init
add → commit
git remote add
  origin 주소
git push origin main
```

GitHub에서 받기

```
git clone 주소
cd 폴더이름
```

매일 반복

```
git pull origin main
(작업)
git status
git add 파일
git commit -m "..."
git push origin main
```

되돌리기

```
git reset HEAD~1
git revert 커밋
git reflog
```